

Chapter 19: Binomial Heaps

1) The "mergeable heaps" data structures support the following operations:-

- 1) MAKE-HEAP() creates and returns a new heap containing no elements.
- 2) INSERT(H, x) inserts node x into heap H , whose Key field has already been filled in.
- 3) MINIMUM(H) returns a pointer to the node in heap H whose Key is minimum.
- 4) EXTRACT-MIN(H) deletes the node from heap H whose Key is minimum. It also returns a pointer to the node.
- 5) UNION(H_1, H_2) creates and returns a new heap that contains all the nodes of heaps H_1 and H_2 . Heaps H_1 and H_2 are destroyed by this operation.
- 6) DECREASE(H, x, K) assigns to node x with heap H the new Key value K , which is assumed to be no greater than its current Key value.
- 7) DELETE(H, x) deletes node x from heap H .

→ Running times for operations on three implementations of mergeable heaps.

Procedure	Binary heap (worst case)	Binomial heap (worst case)	Fibonacci heap (amortized)
MAKE-HEAP	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
INSERT	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(1)$
MINIMUM	$\Theta(1)$	$\Theta(\lg n)$	$\Theta(1)$
EXTRACT-MIN	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(\lg n)$

UNION	$\Theta(n)$	$\mathcal{O}(\lg n)$	$\Theta(1)$
DECREASE-KEY	$\Theta(\lg n)$	$\Theta(\lg n)$	$\Theta(1)$
DELETE	$\Theta(\lg n)$	$\Theta(\lg n)$	$\mathcal{O}(\lg n)$

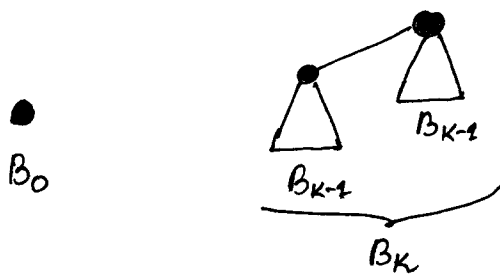
n is the number of items in the heap at the time of the operation.

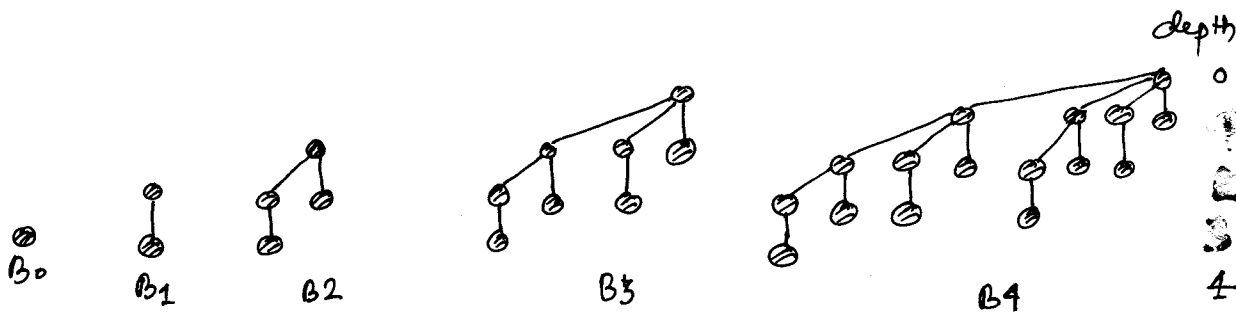
→ If we don't need the UNION operation ordinary binary heaps work well. But if UNION operation must be supported binary heaps perform poorly.

→ Binary, binomial and Fibonacci heaps are all inefficient in their support of the operation SEARCH. For this reason, operations such as DECREASE-KEY and DELETE that refer to a given node require a pointer to that node as part of their input.

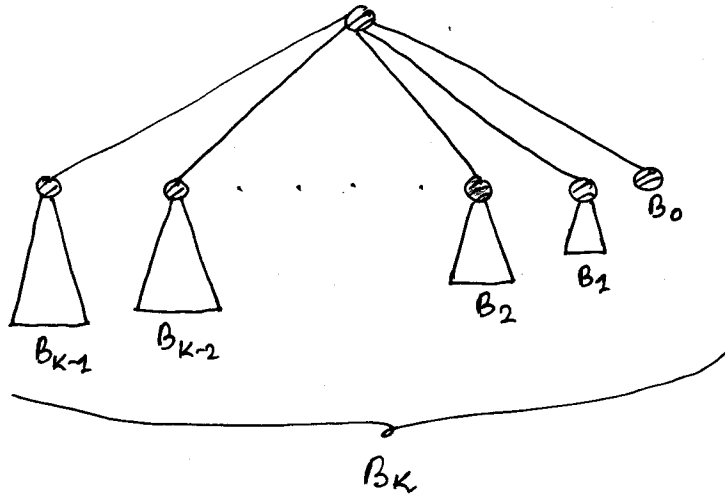
2) Binomial heaps: A binomial heap is a collection of binomial trees.

Binomial tree: The binomial tree B_k is an ordered tree defined recursively. The binomial tree B_0 consists of a single node. The binomial tree B_k consists of two binomial trees B_{k-1} that are linked together such that root of one is the leftmost child of the root of the other.





Another way of looking at binomial tree B_K :-



Properties of binomial trees:-

For the binomial tree B_K :-

- 1) there are 2^K nodes
- 2) the height of the tree is K
- 3) there are exactly $\binom{K}{i}$ nodes at depth i for $i = 0, 1, 2, \dots, K$
- 4) the degree of the root is K , which is greater than that of any other node. Moreover, if the children of the root are numbered from left to right by $K-1, K-2, \dots, 0$, child i is the root of the subtree B_i .

Proof: The proof is by induction on K . For each property the basis is the binomial tree B_0 . Verifying that each property holds for B_0 is $\square$

trivial. For the inductive step we assume that the lemma holds for B_{K-1} .

1) Binomial tree B_K consists of two copies of B_{K-1} and so B_K has $2^{K-1} + 2^{K-1} = 2^K$ nodes.

2) Because of the way the two copies of B_{K-1} are linked to form B_K , the maximum depth of a node in B_K is one greater than the maximum depth in B_{K-1} . By the inductive hypothesis, this maximum depth is $(K-1) + 1 = K$.

3) Suppose $D(K, i)$ is the number of nodes at depth i of binomial tree B_K . Since B_K is composed of two copies of B_{K-1} linked together, a node at depth i in B_{K-1} appears twice in B_K : once at depth i and once at depth $i+1$.

Therefore, the no. of nodes at depth i in B_K is the sum of the no. of nodes at depth i in B_{K-1} and the no. of nodes at depth $i-1$ in B_{K-1} .

$$\text{Thus, } D(K, i) = D(K-1, i) + D(K-1, i-1)$$

$$= \binom{K-1}{i} C + \binom{K-1}{i-1} C = \binom{K}{i} C$$

4) The only node with greater degree in B_K than in B_{K-1} is the root, which has one more child than in B_{K-1} . Since the root of B_{K-1} has degree $K-1$, the root of B_K has degree K . The children of the root of B_{K-1} are roots of $B_{K-2}, B_{K-3}, \dots, B_0$. When B_{K-1} is linked to B_{K-1} , by induction, the children of the resulting root are roots of $B_{K-1}, B_{K-2}, \dots, B_0$.

Corollary: The maximum degree of any node in an n -node binomial tree is $\lg n$.

→ Since the $K;C$ are the binomial coefficients, we use the term binomial tree.

3) Binomial Heaps: A binomial heap H is a set of binomial trees that satisfies the following "binomial-heap properties": -

1) Each binomial tree in H satisfies the min-heap property. Each binomial tree is then said to be min-heap-ordered.

2) For any non-negative integer k , there is at most one binomial tree B_k .

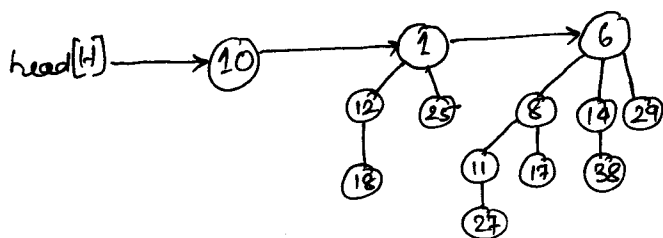
→ The implication of the second property of binomial heaps is that an n -node binomial heap H consists of at most $\lg n + 1$ binomial trees. The binary representation of integer n has $\lg n + 1$ bits

$n_{10} = \langle b_{\lg n}, b_{\lg n-1}, \dots, b_1, b_0 \rangle_2$ so that

$$n = b_0 \cdot 2^0 + b_1 \cdot 2^1 + \dots + b_{\lg n} \cdot 2^{\lg n} = \sum_{i=0}^{\lg n} b_i 2^i$$

The binomial tree B_i appears in H if and only if the bit $b_i = 1$. Hence, binomial heap H contains at most $\lg n + 1$ binomial trees.

eg:- A binomial heap with 13 nodes. $13_{10} = \langle 1101 \rangle_2$.
 $13 = 1 + 4 + 8$. Hence H contains B_0, B_2, B_3 .



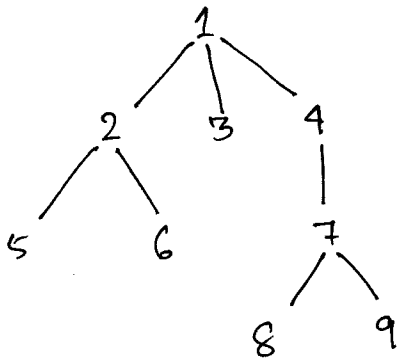
4) **Multway tree**: A tree with any number of children for each node.

Binary tree representation of rooted trees:
(Left-Child, Right-Sibling (LCRS) representation or First-Child, Next-Sibling (FCNS) representation):

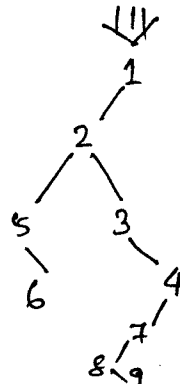
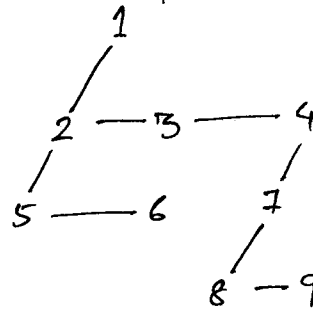
A way to represent a multway tree as a binary tree that uses $O(n)$ space for an n -node rooted tree. $root[1]$ points to the root of the tree. Each node x has the following fields:-

- 1) $Key[x]$
- 2) $parent[x]$
- 3) $left-child[x]$: points to the leftmost child of node x in the multway tree
- 4) $right-sibling[x]$: points to the sibling of x immediately to the right (in the multway tree).

eg:- A Multway tree:-



Binary tree representation:-



Representing binomial heaps:-

Binomial heaps can be represented using the left-child, right-sibling representation. Each node x here also contains the field $\text{degree}[x]$ which is the no. of children of x . If node x is a root parent $\text{parent}[x] = \text{NIL}$.

The roots of the binomial trees within a binomial heap are organized in a linked list, which is called the "root list". Since the roots of the binomial trees do not siblings the sibling field of the root nodes can be used to maintain the linked structure of the root list (which would otherwise be NIL).

The degrees of the roots strictly increase as we traverse the root-list. $\text{head}[H]$ points to the first root in the root-list of H . If binomial-heap H has no elements $\text{head}[H]$ is NIL.

5) CREATING A NEW BINOMIAL HEAP:

The MAKE-BINOMIAL-HEAP procedure makes an empty binomial heap by allocating and returning an object H , such that $\text{head}[H] = \text{NIL}$. The running time is $\Theta(1)$.

6) Finding the minimum Key:

Since a binomial heap is a set of MIN-heap-ordered binomial trees, the minimum key of every tree is at the root. The BINOMIAL-HEAP-MINIMUM procedure checks all the roots. Because there are at most $\lg n + 1$ roots the running time is $O(\lg n)$.

BINOMIAL-HEAP-MINIMUM(H)

1 min- $\text{ptr} \leftarrow \text{NIL}$ $\triangleright$ holds pointer to the node with min. Key
2 $\text{trav} \leftarrow \text{head}[H]$

7

```

3  min-Key  $\leftarrow \infty$ 
4  while  $\text{root} \neq \text{NIL}$ 
5      do if  $\text{Key}[\text{root}] < \text{min-Key}$ 
6          then  $\text{min-Key} \leftarrow \text{Key}[\text{root}]$ 
7               $\text{min-ptr} \leftarrow \text{root}$ 
8       $\text{root} \leftarrow \text{sibling}[\text{root}]$ 
9  return  $\text{min-ptr}$ 

```

4) Uniting two binomial heaps:-

The BINOMIAL-^{LINK}~~HEAP-UNION~~ procedure writes the root of two binomial trees of same order by making one the left-most child of the root of the other. Since it makes the root of a B_{k-1} tree the left-most child of the root of another B_{k-1} tree, the resultant is a B_k tree.

BINOMIAL-LINK(x, y)

```

1  parent[x]  $\leftarrow y$ 
2  sibling[x]  $\leftarrow \text{child}[y]$ 
3  child[y]  $\leftarrow x$ 
4  degree[y]  $\leftarrow \text{degree}[y] + 1$ 

```

The procedure runs in time $O(1)$.

→ The procedure BINOMIAL-HEAP-UNION(H_1, H_2) writes the binomial heaps H_1 and H_2 and returns the resulting heap H . It destroys the representations of H_1 and H_2 in the process.

→ The BINOMIAL-HEAP-UNION procedure has two phases. The first phase is performed by a call to BINOMIAL-HEAP-MERGE that merges the root lists of binomial heaps H_1 and H_2 into a single linked list H that is sorted by degree into monotonically increasing order. If the root lists of H_1 and H_2 have m roots altogether, BINOMIAL-HEAP-MERGE runs in $O(m)$ time by repeatedly examining the roots at the heads of the two root lists and appending the root with the lower degree to the O/P list, removing it from its I/P list in the process.

The resulting root list may have a maximum of two roots of each degree, so that the 2nd phase links roots of equal degree until there is at most one root of ^{each} ~~equal~~ degree.

→ Throughout the BINOMIAL-HEAP-UNION procedure we maintain three pointers into the root list:-

- 1) x points to the root currently being examined
- 2) $\text{prev-}x$ points to the root preceding x on the root list
- 3) $\text{next-}x$ points to the root following x on the root list

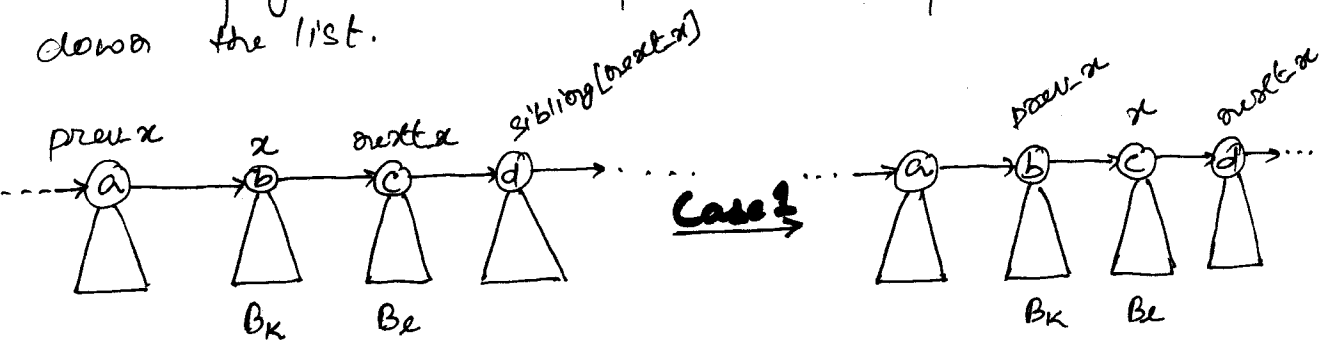
The BINOMIAL-HEAP-MERGE (code not given) guarantees that if two roots in H have the same degree, they are adjacent in the root list.

During the execution of BINOMIAL-HEAP-UNION there may be three roots of the same degree appearing on the root list H at some time

→ The following 4 cases arise during the execution of BINOMIAL-HEAP-UNION:-

Case 1: Case 1 occurs when $\text{degree}[x] \neq \text{degree}[\text{next-}x]$. Thus if x is the root of a B_k -tree and $\text{next-}x$ is the root of a $B_{k'}$ -tree then $k \neq k'$. We do not need to perform any linking since the trees have different orders. 9

We simply march the pointers one position farther down the list.

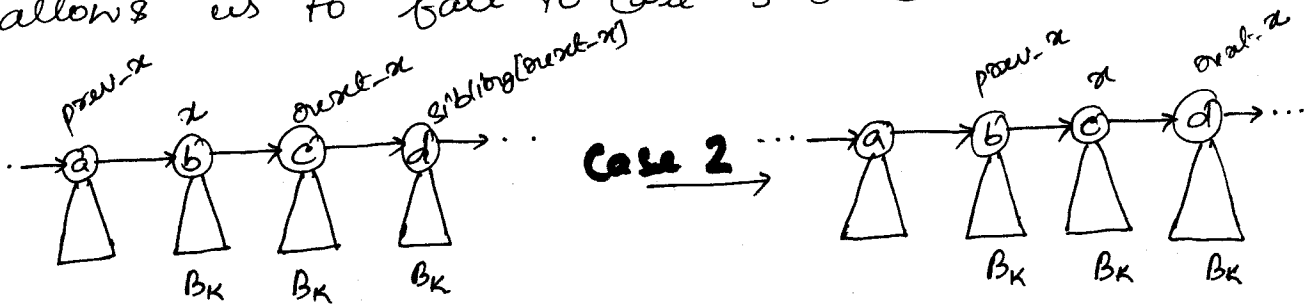


Case 2: Case 2 occurs when x is the first of three roots of equal degree, that is, when:—

$$\text{degree}[x] = \text{degree}[\text{next-}x] = \text{degree}[\text{sibling}[\text{next-}x]]$$

We cannot simply link x to $\text{next-}x$ or $\text{next-}x$ to x since the resulting binomial tree would have a degree one greater than degree of sibling of $\text{next-}x$. We actually need to link $\text{next-}x$ and

sibling $[\text{next-}x]$. In case 2 we just march the pointers one position farther down the list, which allows us to fall to case 3 or case 4.



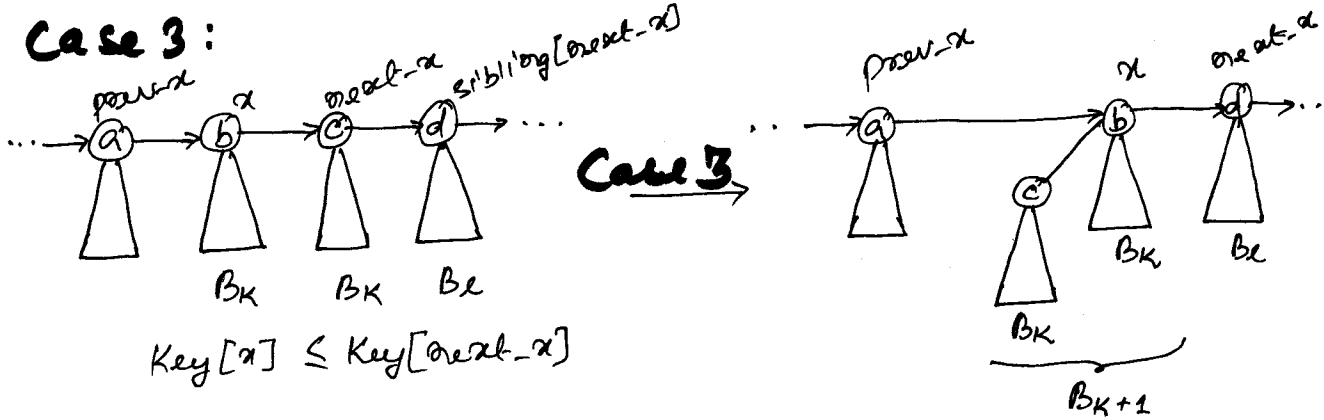
In cases 3 and 4 we have:—

$$\text{degree}[x] = \text{degree}[\text{next-}x] \neq \text{degree}[\text{sibling}[\text{next-}x]]$$

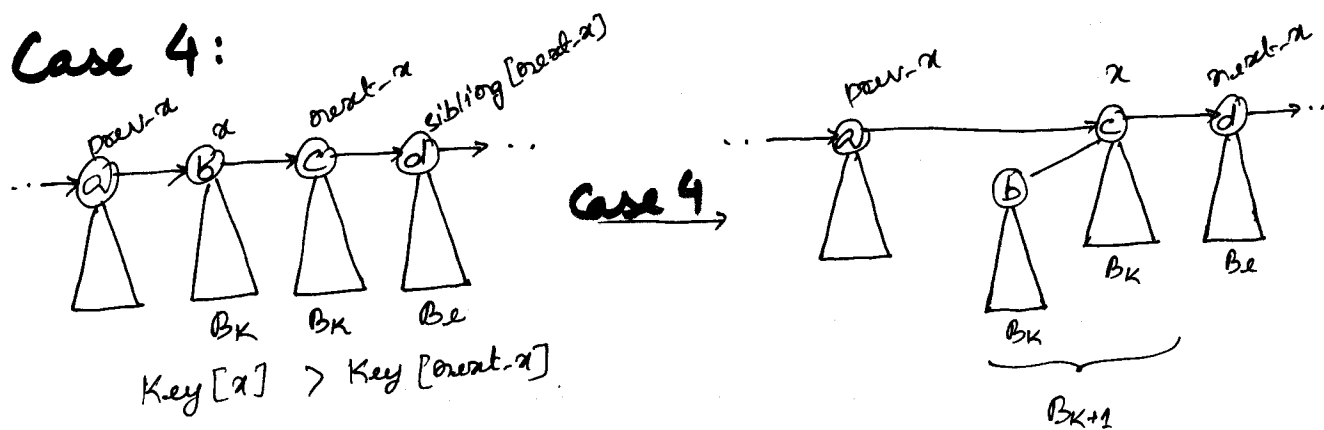
If $\text{Key}[x] \leq \text{Key}[\text{next-}x]$ we link $\text{next-}x$ to x so that the resulting tree is union-heap ordered.

If $\text{Key}[x] > \text{Key}[\text{next-}x]$ we link x to $\text{next-}x$.

Case 3:



Case 4:



→ In either case 3 or case 4 we have just linked 2 B_K trees to form a B_{K+1} tree, which x now points to. There were already zero, one or two other B_{K+1} trees on the root list so that x becomes the first of either one, two or three B_{K+1} trees on the root list. If x is the only one then we enter case 1 in the next iteration. If x is the first of two then we enter either case 3 or case 4. It is when x is the first of three that we enter case 2 in the next iteration.

BINOMIAL-HEAP-UNION(H_1, H_2)

```
1   $H \leftarrow \text{MAKE\_BINOMIAL\_HEAP}()$ 
2   $\text{head}[H] \leftarrow \text{BINOMIAL\_HEAP\_MERGE}(H_1, H_2)$ 
3  free the objects  $H_1$  and  $H_2$  but not the lists they point to
4  if  $\text{head}[H] = \text{NIL}$   $\triangleright$  trying to unite 2 empty heaps
5      then return  $H$ 
6   $\text{prev-}x \leftarrow \text{NIL}$ 
7   $x \leftarrow \text{head}[H]$ 
8   $\text{next-}x \leftarrow \text{sibling}[x]$ 
9  while  $\text{next-}x \neq \text{NIL}$ 
10     do if ( $\text{degree}[x] \neq \text{degree}[\text{next-}x]$ ) or
        ( $\text{sibling}[\text{next-}x] \neq \text{NIL}$  and  $\text{degree}[\text{sibling}[\text{next-}x]] = \text{degree}[x]$ )
11         then  $\text{prev-}x \leftarrow x$   $\triangleright$  Cases 1 and 2
12              $x \leftarrow \text{next-}x$   $\triangleright$  Cases 1 and 2
13     else if  $\text{Key}[x] \leq \text{Key}[\text{next-}x]$ 
14         then  $\text{sibling}[x] \leftarrow \text{sibling}[\text{next-}x]$   $\triangleright$  3
15              $\text{BINOMIAL\_LINK}(\text{next-}x, x)$   $\triangleright$  3
16     else if  $\text{prev-}x \neq \text{NIL}$   $\triangleright$  4
17         then  $\text{sibling}[\text{prev-}x] \leftarrow \text{next-}x$ 
18         else  $\text{head}[H] \leftarrow \text{next-}x$ 
19              $\text{BINOMIAL\_LINK}(x, \text{next-}x)$   $\triangleright$  4
20              $x \leftarrow \text{next-}x$   $\triangleright$  4

21      $\text{next-}x \leftarrow \text{sibling}[x]$ 
22 return  $H$ 
```

→ The run time of ~~$O(\lg n)$~~ **BINOMIAL-HEAP-UNION** is $O(\lg n)$ where n is the total no. of nodes in H_1 and H_2 .
 Let H_1 contain n_1 nodes and H_2 contain n_2 nodes so that $n = n_1 + n_2$. H_1 contains at most $\lfloor \lg n_1 \rfloor + 1$ roots and H_2 contains $\lfloor \lg n_2 \rfloor + 1$ roots at max. Hence H contains at most $\lfloor \lg n_1 \rfloor + \lfloor \lg n_2 \rfloor + 2 \leq 2 \lfloor \lg n \rfloor + 2 = O(\lg n)$ roots, immediately after the call to **BINOMIAL-HEAP-MERGE**.

Each iteration of the while loop takes $O(1)$ time and there are at most $\lfloor \lg n_1 \rfloor + \lfloor \lg n_2 \rfloor + 2 = O(\lg n)$ iterations because each iteration either advances the pointers one position down the list or removes a root from the root list. The total time is thus $O(\lg n)$.

8) Inserting a node:

BINOMIAL-HEAP-INSERT(H, x)

- 1 $H' \leftarrow \text{MAKE-BINOMIAL-HEAP}()$
- 2 $\text{parent}[x] \leftarrow \text{NIL}$
- 3 $\text{child}[x] \leftarrow \text{NIL}$
- 4 $\text{sibling}[x] \leftarrow \text{NIL}$
- 5 $\text{degree}[x] \leftarrow 0$
- 6 $\text{head}[H'] \leftarrow x$
- 7 $H \leftarrow \text{BINOMIAL-HEAP-UNION}(H, H')$

Runtime = $O(\lg n)$

9) Extracting the node with minimum key:

BINOMIAL-HEAP-EXTRACT-MIN(H)

1. bind the root x with the minimum Key in the root list of H , and remove x from the root list of H
2. $H' \leftarrow \text{MAKE_BINOMIAL_HEAP}()$
3. reverse the order of the linked list of x 's children and set $\text{head}[H']$ to point to the head of the resulting list
4. $H \leftarrow \text{BINOMIAL_HEAP_UNION}(H, H')$
5. return x

Running time = $O(\lg n)$.

10) Decreasing a Key:

BINOMIAL-HEAP-DECREASE-KEY(H, x, K)

1. if $K > \text{Key}[x]$
2. then error "new Key is greater than current Key"
3. $\text{Key}[x] \leftarrow K$
4. $\text{trav} \leftarrow x$
5. $\text{parent-trav} \leftarrow \text{parent}[\text{trav}]$
6. while $\text{parent-trav} \neq \text{NIL}$ and $\text{Key}[\text{trav}] < \text{Key}[\text{parent-trav}]$
7. do exchange $\text{Key}[\text{trav}] \leftrightarrow \text{Key}[\text{parent-trav}]$
8. D exchange the Satellite fields too
9. $\text{trav} \leftarrow \text{parent-trav}$
10. $\text{parent-trav} \leftarrow \text{parent}[\text{trav}]$

The procedure decreases a Key in the same manner as in a binary min-heap: by "bubbling up" the Key in the heap.

Since the maximum depth of a binomial tree can be $\lg n$ for a binomial heap of n nodes, the no. of iterations of the while loop is $O(\lg n)$.

12) Deleting a Key:

BINOMIAL-HEAP-DELETE(H, x)

- 1 BINOMIAL-HEAP-DECREASE-KEY($H, x, -\infty$)
- 2 BINOMIAL-HEAP-EXTRACT-MIN(H)

It is assumed that no node in the binomial heap has the value $-\infty$ initially.

Running time: $O(\lg n)$.